DESARROLLO WEB

Gustavo Tabares

EJE 2

Analicemos la situación



Fuente: Adobe/332711195

AREANDINA
Fundación Universitaria del Área Andina

ÍNDICE

Introducción 3
Arquitectura web 4
Patrones de desarrollo. 9
Modelo-vista-controlador MVC. 9
Data Access Object - DAO 17
Mapeo Objeto Relacional (ORM) 18
Bibliografía 20

INTRODUCCIÓN

El desarrollo de aplicaciones comerciales, hoy en día para los desarrolladores exigen tener más habilidades, creatividad e innovación para conocer e integrar tecnologías, que les permitan desarrollar aplicaciones orientadas a resolver problemas complejos que satisfagan los problemas de concurrencia y acceso por parte de los usuarios. El desarrollo de aplicaciones implementando arquitecturas de desarrollo basadas en capas, permite organizar, separar y asignar responsabilidades a cada capa. La capa de presentación ofrece al usuario una vista que le permite realizar peticiones a través de la interacción desde el navegador, permitiendo a la capa de control aplicar las reglas y procesos basados en métodos de negocio para posteriormente la capa de datos almacene la información basada en la lógica de negocio, el desarrollo de esta estructura es posible con la implementación de una arquitectura basada en capas y una de ellas es el Modelo-Vista-Controlador. Este eje fortalece el nodo Areandino de Creatividad, innovación y emprendimiento, para crear y generar ideas y proyectos a través de la resolución de problemas contando con el análisis de datos del entorno en todas las disciplinas y la ideación de modelos de empresa en torno a dichas soluciones.

Arquitectura web



Figura 1. Arquitectura web
Fuente: Adobe/473748577

La arquitectura web está basada en: arquitectura cliente-servidor, donde el cliente es el que interactúa y establece una comunicación a través del navegador, encargado de interpretar y desplegar la información suministrada por el servidor.

El servidor está atento a la solicitud de peticiones realizadas por el cliente para visualizar las páginas solicitadas o procesar la información suministrada por el cliente.

La interacción entre el cliente y el servidor se da a través de una comunicación por medio de protocolo de comunicación que puede ser de dos tipos:

- **Http:** este protocolo se encarga de intercambiar la información en hipertexto.

- **Https:** este protocolo es una extensión de HTTP y se encarga de la seguridad del sitio a través de un canal cifrado, la información es encriptada, logrando así una comunicación segura entre el cliente y el servidor. Garantiza la privacidad y seguridad de la información, se puede visualizar gráficamente a través de un candado en el navegador.

HTTP:
Protocolo de Transferencia de Hipertexto, es un protocolo de navegación de la capa de presentación para la transferencia de documentos.

Las aplicaciones web han aumentado considerablemente frente a otros desarrollos de tipo escritorio. El uso de la internet, la facilidad de acceso a través de los navegadores, la actualización y la incorporación de nuevos componentes para las aplicaciones web son mucho más fáciles sin contar con la intervención del usuario.

Las páginas web son el componente principal de una aplicación o sitio web, los navegadores solicitan páginas almacenadas o creadas dinámicamente con información a los servidores web, las aplicaciones web contiene código HTML y JavaScript dinámicos que son ejecutados desde el servidor antes de ser cargados en el navegador. Las páginas dinámicas están agrupadas y desarrolladas lógicamente para responder a las peticiones del usuario, los componentes de una aplicación web son:

- Lógica de negocio
  - Es el elemento importante de la aplicación
  - La definición de los procesos que integra la aplicación.
  - Las operaciones, reglas requeridas para proveer el servicio
- Administración de datos
  - La conexión a la base de datos
  - La gestión de la base de datos y los archivos
- Interfaz de usuario
  - Las formas o vistas para acceder a través de navegadores.
  - Funcionalidad y accesibilidad desde el navegador

Las aplicaciones web se diseñan mediante el modelo por capas, los hay de modelo de dos capas y modelo de n capas.



Instrucción

Para ampliar la información se invita al estudiante a ingresar a la página principal para revisar el recurso de aprendizaje:

Animación



Fat client:

Un cliente pesado en una red con muchos programas o recursos cargados locamente con poca dependencia de los recursos de la red.

Modelo de dos capas

La aplicación se ejecuta en mayor parte del lado del cliente (fat client).

Las capas que la integran son:

- Cliente: el desarrollo de la aplicación integra la lógica del negocio.
- Servidor: se encarga de administrar los datos.

Este modelo presenta muchas limitaciones, como difícilmente escalable, la conexión a la base de datos es reducida, no responde correctamente a la alta carga de la red presentando una flexibilidad reducida lo que da una funcionalidad limitada, la figura 2 muestra un modelo de dos capas.

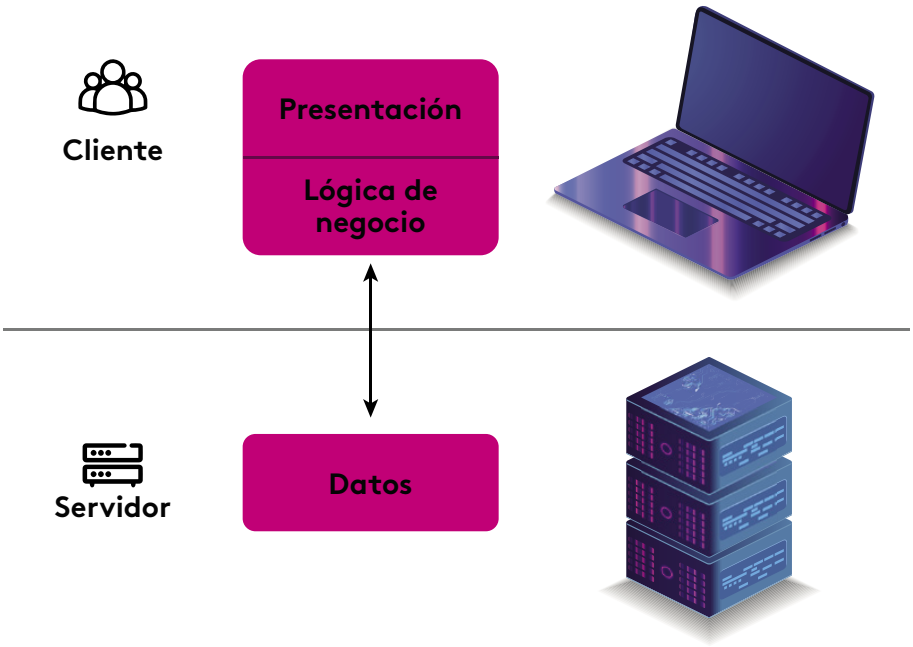


Figura 2. Modelo de dos capas
Fuente: propia

Modelo de tres capas

Fue diseñada para suplir las limitaciones que presenta el modelo de dos capas, incluyendo una capa de proceso entre la capa presentación y la capa de datos, manejando los procesos de forma separada a la interfaz y los datos, esta capa permite concentrar la lógica de negocio, las aplicaciones web actuales implementan este modelo de tres capas.

Las capas del modelo son:

- La capa de presentación (permite la interacción del cliente con la aplicación).
- La capa de proceso (servidor web).
- La capa de datos (servidor de datos).

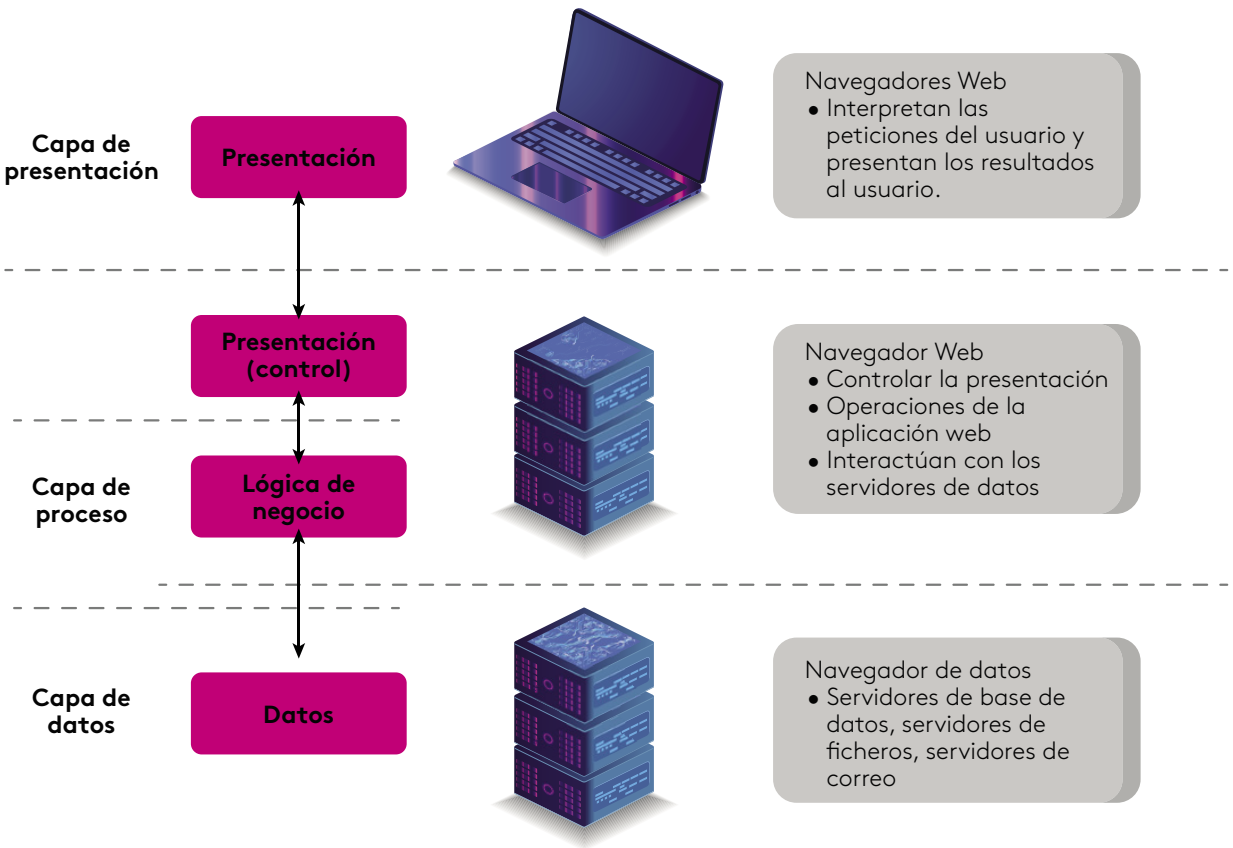


Figura 3. Modelo de dos capas
Fuente: propia

Lectura recomendada

Para ampliar este apartado se invita al estudiante desde la página principal a realizar la lectura complementaria:

Patrones Arquitectónicos de Aplicaciones Empresariales

Roberto García González

Patrones de desarrollo

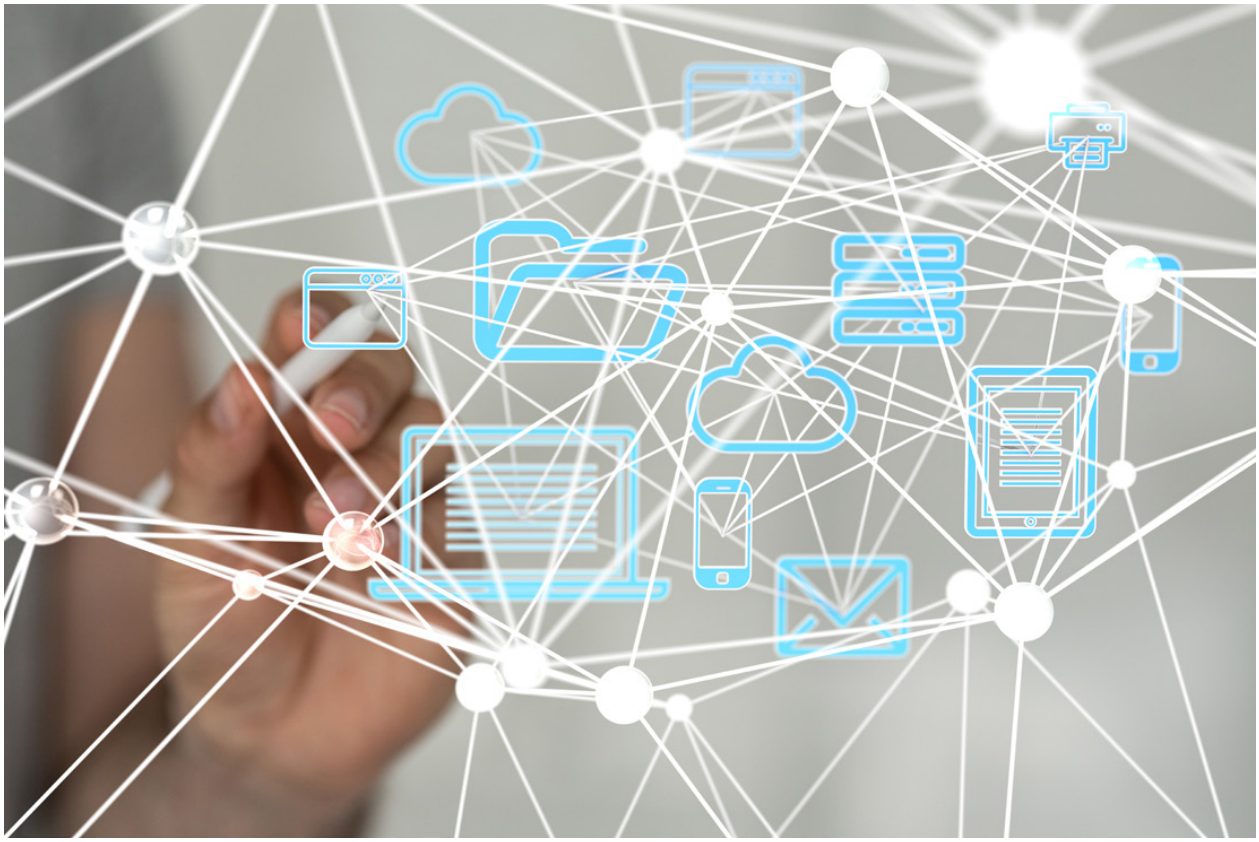


Figura 4. Patrones de desarrollo
Fuente: Adobe/167912120

Modelo-vista-controlador MVC

El MVC, hace parte de un modelo arquitectónico que surgió en los años 70 en Xerox Park por Trygve Reenskaug como librería de clases del lenguaje de programación Small-talk-80. Con la finalidad de cerrar la brecha entre el modelo mental del usuario humano y el modelo digital que existe en la computadora.

MVC define un patrón de desarrollo por capas basado en tres componentes distintos que son: el modelo, la vista y el controlador. Organizando los componentes de un lado, que hacen parte de la presentación de la información visual, otra capa se encarga de la interacción y control de las peticiones y respuestas que genera el usuario y la capa final encargada de la lógica de los datos.

La reutilización de código y la separación de conceptos, es el factor fundamental de este patrón de arquitectura, que facilita el desarrollo y mantenimiento de las aplicaciones, en la siguiente figura muestra el esquema del patrón MVC.

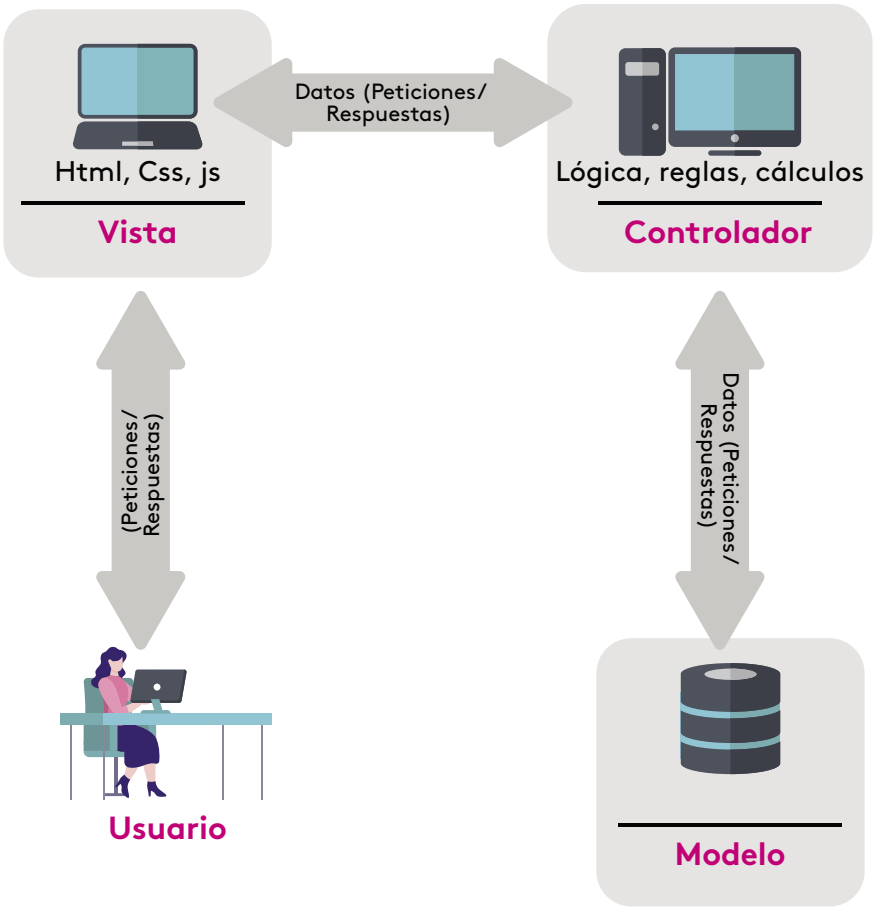


Figura 5. Patrón arquitectónico MVC
Fuente: propia

Instrucción

Les invitamos a afianzar sus conocimientos ingresando a la página principal para revisar el recurso de aprendizaje:

Videorresumen

El patrón MVC se compone de los siguientes elementos:

Modelo

La capa modelo es la encargada de la lógica del negocio, representada en el almacenamiento de la información en bases de datos o archivos de almacenamiento con la que funciona la aplicación, es la encargada de validar y gestionar las operaciones básicas que se realizan con los datos almacenados a través de un CRUD (Create, Read, Update, Delette).

Los objetos del modelo están directamente relacionados con la interacción de la base de datos de la aplicación, que por lo general de aquí parten los conceptos y la lógica en los cuales se desea implementar la aplicación.

Las funciones asignadas al modelo son:

- Gestionar el almacenamiento de los datos.
- Establece la funcionalidad de la aplicación basado en las reglas del negocio.
- Notificar los cambios en los datos almacenados.

Vista

Corresponde a la capa de presentación con la que interactúa el usuario, es el encargado de representar la información que envía el modelo por medio del controlador en un formato claro.

La vista permite visualizar los datos del modelo, estando apartada de los objetos del modelo, es el encargado del uso de los datos para ser visualizados en una interfaz de presentación para el usuario.

La vista construye interfaces de usuario para que esta interactúe con la aplicación, la integran objetos como botones, objetos gráficos que conforman la aplicación.

Las tecnologías que se integran para construir la vista son HTML, hojas de estilos en cascadas CSS y librerías de Javascript, que también puede estar diseñadas bajo plantillas template.

La capa de la vista no es solamente HTML o texto derivado de los datos, estos pueden ser representados en diferentes formatos de acuerdo con la necesidad del usuario y pueden ser en vídeos, sonido, documentos de texto y algún otro formato.

La vista tiene las siguientes funciones:

- Recibir las respuestas del controlador o el modelo.
- Desplegar la información en la interfaz del usuario.

Controlador

Su función principal es recibir las peticiones de la vista y transmitirlas al modelo, hace las veces de intermediario.

Se encarga de enviar y recibir los datos solicitados por el controlador desde un proceso del sistema, invocando los métodos necesarios en el modelo y generando las respuestas con los datos solicitados desde la aplicación.

El controlador establece una comunicación con el modelo para recibir los datos necesarios y pasarlos a la vista, donde no tiene ninguna injerencia de cómo se muestran al usuario pues de esto se encarga la vista como lo indica (Sommerville, 2006).

El controlador presenta las siguientes funciones:

- Recibe las peticiones de la vista.
- Se encarga de los eventos y hace peticiones al modelo y a la vista.

La capa modelo representa los datos del dominio, basados en la lógica del negocio, donde se almacenará la información en las entidades o variables del sistema que se está desarrollando.

Como lo indica el siguiente ejemplo, una aplicación de facturación, el modelo estaría conformado por la clase factura, cliente o proveedor.

Cuando la estructura de la aplicación está definida por un sistema distribuido, consumiendo servicios de otras aplicaciones, en el modelo se hace necesario contar con las clases DTO Data Transfer Objects, clases de transferencia de datos, para intercambiar información entre ellas.

Como se mencionó, en esta capa encontramos la lógica del negocio de la aplicación, donde se deben establecer las reglas, condiciones, restricciones y acciones que deben cumplir, la información al ser almacenada en las entidades del dominio, teniendo como responsabilidad mantener la integridad del sistema.

El modelo también se encarga de gestionar el almacenamiento y retorno de los datos y entidades del dominio, basado en unidades de persistencia delegadas a motores de bases de datos, implementando componentes DAL Data Access Layer, capa de acceso a datos y ORMs que se encargan del acceso a datos desde los lenguajes de programación orientados a objetos.

ORM:
Mapeo de objetos relacionales

Instrucción

Les invitamos a afianzar sus conocimientos ingresando a la página principal para revisar la actividad de aprendizaje:

Caso simulado

Ejemplo

Veamos un ejemplo de cómo se implementa MVC en PHP.

1. Partimos de crear una estructura de carpetas:
   - Controlador.
   - Db.
   - Modelo.
   - Vista.

Index.php

```
<?php
require_once("db/db.php");
require_once("controlador/persona_controlador.php");
?>
```

Db.php

```
<?php
class Conectar{
    public static function conexion(){
        $conexion=new mysqli("localhost", "root", "", "mvc");
        $conexion->query("SET NAMES 'utf8'");
        return $conexion;
    }
}
?>
```

Ejemplo

Modelo/Persona_modelo.php

```
<?php
class persona_modelo{
    private $db;
    private $persona;

    public function __construct(){
        $this->db=Conectar::conexion();
        $this->persona=array();
    }
    public function get_persona(){
        $consulta=$this->db->query("select * from persona;");
        while($filas=$consulta->fetch_assoc()){
            $this->persona[]=$filas;
        }
        return $this->persona;
    }
}
?>
```

Controlador/persona_controlador.php

```
<?php
//Llamada al modelo
require_once("model/Persona_modelo.php");
$per=new persona_modelo();
$datos=$per->get_persona();

//Llamada a la vista
require_once("vistas/persona_vista.phtml");
?>
```

Ejemplo

Vista/persona_vista.phtml

```
<!DOCTYPE html>
<html lang="es">
    <head>
        <meta charset="UTF-8" />
        <title>Personas</title>
    </head>
    <body>
        <?php
            foreach ($datos as $dato) {
                echo $dato["nombre"]."<br/>";
            }
        ?>
    </body>
</html>
```

Instrucción

Les invitamos a afianzar sus conocimientos ingresando a la página principal para revisar la actividad de aprendizaje:

Práctica

Esquema gráfico el MVC en PHP

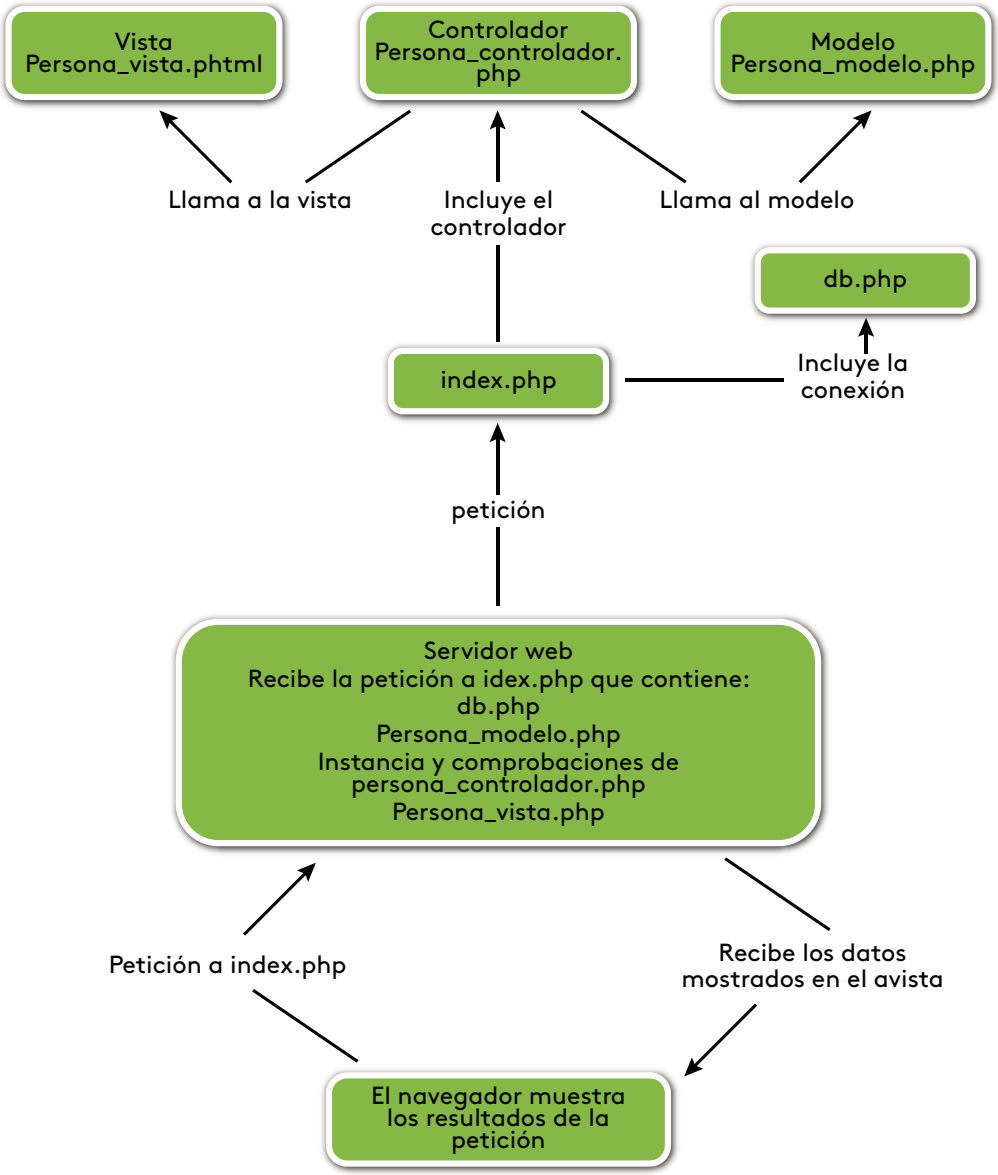


Figura 6. Esquema MVC
Fuente: propia

Video

Para ampliar la información se invita al estudiante a ingresar a la página principal para visualizar la siguiente videocápsula:

Ejemplo de MVC con PHP Parte 1 de 8 | Curso PHP y MySQL

https://youtu.be/BEyNt7PAK0E

Data Access Object - DAO

Cuando se desarrollan aplicaciones web con acceso a datos, es necesario definir algún método, por persistencia (Hibernate, JDO, iBatis), JDBC, un Fichero de Texto o LDAP, etc. Cuando tenemos varias opciones para acceder a los datos puede ser un problema, la forma de acceder a los datos muchas veces depende del fabricante y a la forma de almacenamiento que utilicemos.

El Data Access Object o patrón de diseño DAO, permite resolver este problema abstrayendo y encapsulando el acceso a datos. El DAO se encarga de manejar la conexión con la fuente de datos, obteniendo y guardando de forma persistente los datos.

A través del DAO este realizan operaciones de forma automáticas hacia la BD, no son necesarias las transacciones, como ejemplos podemos mencionar las búsquedas por ID, los procesos de creación, actualizado y borrado de registros, consultas u otras operaciones que se realicen a menudo en la aplicación.

Hibernate:

Es una framework para el mapeo objeto relacional ORM en java, en .net se llama NHibernate.

JDO:

Java Data Objects: permite reducir el código SQL detallado y gestiona las transacciones del sistema.

IBatis:

Framework basado en capas se encarga de la capa de persistencia y se encuentra entre la lógica de negocio y la capa de la base de datos.

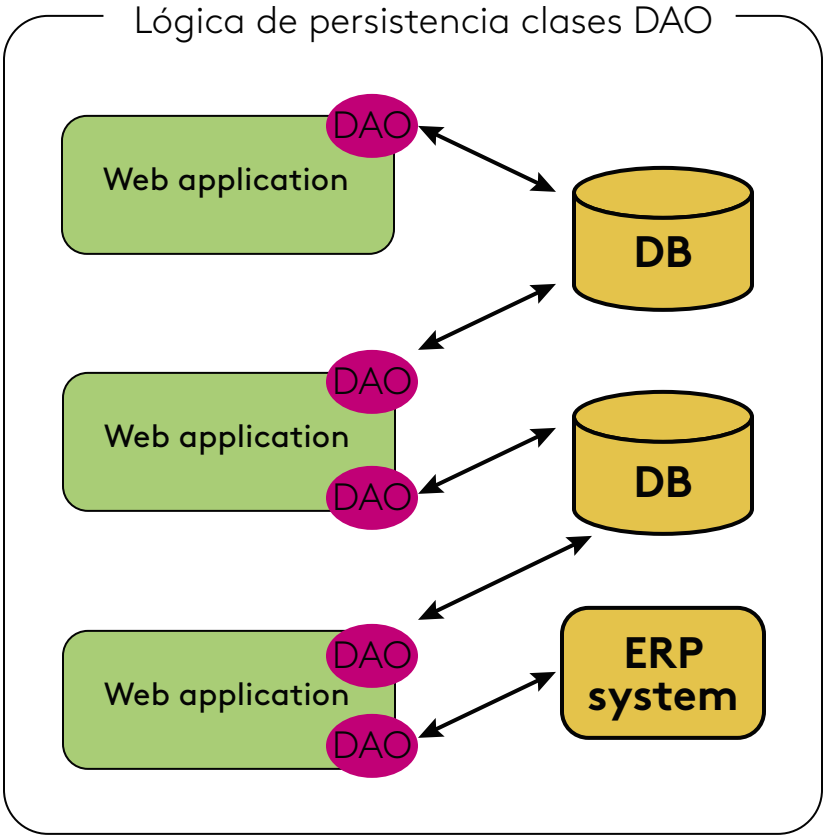


Figura 7. Persistencia DAO
Fuente: propia

Métodos para lograr la persistencia

- **Serialización:** que convierte los objetos en una serie de bits almacenables en disco. Tiene la desventaja de que carece de lenguaje de consulta y no da soporte a altas tasas de acceso concurrente.

- **Java Data Connectivity - JDBC:** permite acceder a sistemas gestores de bases de datos relacionales, aunque su uso está muy extendido, la tendencia es migrar a herramientas más potentes de mapeo de objetos relacional, esta herramienta se basa en delegar el acceso a la base de datos relacionales a un framework, java define el modelo de programación para la persistencia de entidades, pero deja su implementación a proveedores externos, entre los que tenemos IBM con Hibernate.

- **Herramientas de Mapeo Objeto-Relacional (ORM):** se encargan de automatizar la asignación de objetos de un lenguaje a tablas de bases de datos relacionales.

Mapeo Objeto Relacional (ORM)

El desarrollo web en algún lenguaje como C# o Java, se hace uso de objetos especializados para establecer conexión con una base de datos para realizar operaciones básicas necesarias, definiendo objetos de tipo Connection y Query siendo estos conceptos de bases de datos, que son llevados a programas orientados a objetos sin tener ninguna relación con ellos. Igualmente, se generaban consultas en lenguaje SQL para gestionar los datos a través de cadenas de texto u objetos definidos en el lenguaje.

Los modelos de datos relacionales y la programación orientada a objetos generan diferentes modos de pensar y estos deben de comunicarse a través de un código generado por el lenguaje de programación, pero podría ser mucho más complejo, por ejemplo, en POO una factura y su información definen en POO una clase Factura con sus atributos, cuando se desea consultar la información de algún atributo se utiliza Factura. Atributo sin tener en cuenta cómo se relaciona, en la base de datos, se deben modelar y construir varias tablas, una para la factura y otra para los atributos, teniendo en cuenta los índices y relaciones externas en la que se relaciona la información, en el caso de que alguna información de la factura perteneciera a una o más facturas, es necesario definir una tercera tabla que se relacione con las anteriores permitiendo localizar la información, a esto no referimos con diferentes modos de pensar. La figura 3 muestra el mapeo de una base de datos relacional.

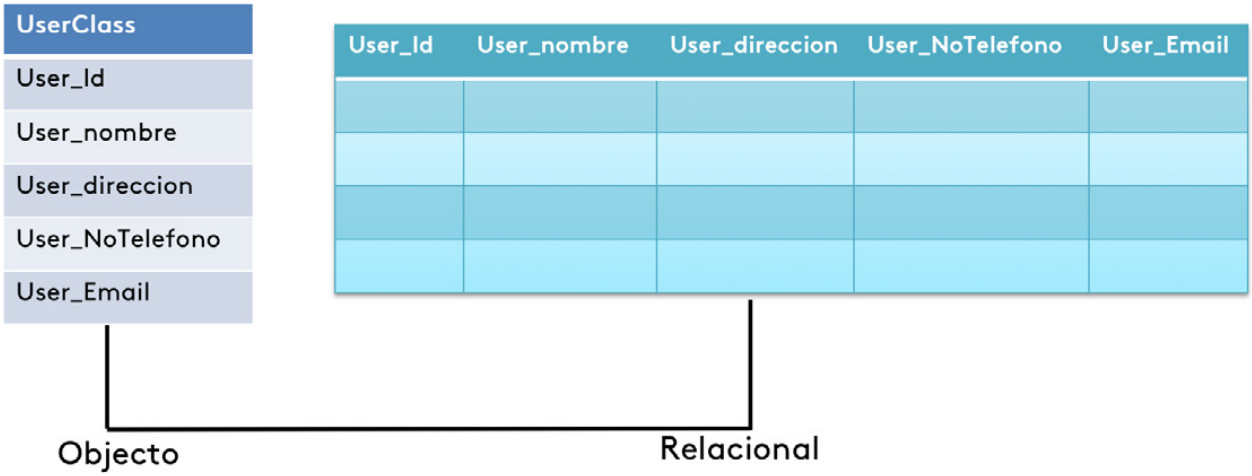


Figura 8. Mapeo relacional
Fuente: propia

La programación puede ser mucho más compleja debido a que las bases de datos cuentan con procedimientos almacenados como pequeños programas estructurados que van dentro de las bases de datos, ejecutando transacciones y otras funciones que son totalmente independientes a la POO.

El ORM no es necesario utilizar SQL, permite generar clases basadas en tablas de una base de datos teniendo en cuenta sus relaciones y viceversa, desde una estructura de clases crear las tablas necesarias en la base de datos, encargándose de todo sin tener que acceder al gestor de la base de datos.

Conclusión

El patrón de diseño MVC se encarga de separar y asignar responsabilidades en el desarrollo de una aplicación web, la mayoría de las aplicaciones web actuales están enfocadas en la implementación de este patrón de diseño, que a la vez ofrece un orden y claridad en desarrollo, permitiendo implementar frameworks e integrar componentes cumpliendo con estándares y buenas prácticas de programación.

BIBLIOGRAFÍA

Anquetil, R. (2019). *Fundamental Concepts for Web Development: HTML5, CSS3, JavaScript and much more! For complete beginners!* Kindle.

Berenguel Gómez, J. (2015). *Desarrollo de aplicaciones web en el entorno servidor*. Paraninfo, S.A.

Burns, E., Schalk, C., & Griffin, N. (2010). *JavaServer faces 2.0 [electronic resource] : the complete reference*. Obtenido de https://biblioteca.areandina.edu.co/cgi-bin/koha/opac-detail.pl?biblionumber=53626&query_desc=su%3A%22Java%20Computer%20program%20language%22

De Caso Parra, A. (2018). *Javascript 2018 (guía práctica)*. Anaya.

Gauchat, J. (2012). *El gran libro de HTML5, CSS3 y JavaScript*. Editorial: Marcombo. Obtenido de https://lc.fie.umich.mx/~rochoa/Manuales/WEB/HTML5_CSS3_JAVASCRIPT.pdf

Grupo Media Active. (2016). *Aprender a programar aplicaciones con HTML, CSS y Javascript*. Editorial: Marcombo.

Niederst, J. (2012). *Learning Web Desing, A Beginner´s guide to HTML, CSS, JavaScript, and web graphics*. O´Reilly.

Recio García, J. (2016). *HTML5, CSS3 y JQuery*. Ra-Ma.

Sommerville, I. (2006). *Ingeneria del Software*. Madrid: Pearson Educación S.A.